Take-away Sheets

Applied AI and Automation for Executives. One page for each session: what you keep.

Session 1

Seeing: pixels, features and neurons

How can a machine read a picture when all it has is numbers?

- SEEING: pixels, features and neurons
- A grey picture is a grid of numbers: 8 x 8 = 64. A colour pixel is 3 numbers (red, green, blue): 8 x 8 x 3 = 192.
- A feature is a measurement from the input, such as the floor area of a house or the ink in the centre of a digit.
- Weighted sum: multiply each input by its weight and add. z = w1x1 + w2x2 + b. House price = 2 x floor area + 10 x bedrooms + 30 (the bias, the base price), in UGX million.
- Neuron: weighted sum + bias, then an activation function. Umbrella: 0.1 x rain + 0.05 x minutes - 6 = 2.5; a step gives 1, carry it. a = f(z). ReLU(z) = max(0, z). Sigmoid: 1/(1 + e^(-z)).
- Without an activation function, stacked layers collapse into one weighted sum. A layer is a table: 16 neurons are 16 rows of weights, a = f(Wx + b).
- One weight learns by the loop: predict, measure the loss, take a gradient descent step: w new = w old - eta x gradient (eta is the learning rate). Loss L = 1/2 (y - prediction)^2; for many examples, MSE = (1/n) sum of squared errors. Gradient here = -x(y - prediction) = −4, so 0.5 - 0.1 x (−4) = 0.9, then 1.14, 1.284. Backpropagation is the chain rule: dL/dw = dL/dprediction x dprediction/dz x dz/dw.
- A network stacks layers: 64 inputs, a hidden layer of 16 neurons, 10 outputs. 1,210 parameters (weights and biases).
- An epoch is one full pass over the training data. In the recorded run, 17% right after 1 epoch and 97% after 200.
- Recorded run, 5 November 2026: 436 of 450 unseen digits right. Shift each digit by 1 pixel: 46% right.
- Ask of any image system: what was it trained on, what was it tested on, and what does a person check?

I keep: A one-page sheet: how a picture becomes numbers, a neuron worked by hand on a house price and an umbrella, and the digit results.

Session 2

Learning from examples: types of machine learning, decision trees, nearest neighbours

How does a machine learn a rule from examples, and how much can I trust what it learned?

- LEARNING FROM EXAMPLES
- Supervised learning: examples with the right answer (the label). Classification predicts a category (spam or not spam); regression predicts a number (a house price).
- A decision tree is yes-or-no questions: splits and leaves; depth = questions on the longest path. It reads as plain rules you can audit.
- Share p = count / group size. 12 days, 4 rained: p rain = 1/3, p dry = 2/3.
- Gini impurity G = 1 - sum of p squared = the chance two random picks disagree. 12 days: 1 - 1/9 - 4/9 = 0.44. 0 = pure, 0.5 = half and half. Lower is purer.
- Split on the sky: cloudy 0.49, clear 0; weighted by size 7/12 x 0.49 + 5/12 x 0 = 0.29; the drop 0.16 is the Gini gain. The tree keeps the question with the largest drop.
- log2 n = how many yes-or-no questions pick one of n equal choices (log2 8 = 3). Entropy H = sum of p x log2(1/p) = average questions to learn the answer: 12 days 0.918. Information gain = drop in entropy: 0.918 - 0.575 = 0.343.
- Overfitting: a tree with no limit learns the slips. Real run: 192 of 192 on claims it saw (100%), 42 of 48 on held-out claims (87.5%). Judge the held-out score.
- k-fold cross-validation: cut into k folds, hold each out once, score it (fold score = right / 48). Five scores 0.9167, 0.9167, 0.9375, 0.9375, 0.9375; mean = (1/k) x sum = 0.929. Report the spread. Best depth: 3 questions, 93.3% (100 fold scores).
- Distance: one feature, the size of the gap |a - b|. Two features: Euclidean = square root of the sum of squared gaps (3 and 4 give 5); Manhattan = sum of the gaps (3 + 4 = 7).
- k-nearest neighbours: average the k nearest (250, 270, 190 gives 710 / 3 = 237) or take a majority vote for a category. Scale first: min-max x′ = (x - min) / (max - min); claims 69% raw, 89% scaled.
- Words: the computer compares shared words (rare words count more), a closeness score from 0 to 1; Session 4 shows how.
- Recorded runs, 5 and 11 November 2026: on 40 customer messages the learned models scored about 55% with wide swings; a simple keyword rule got 23 of 40. Forty examples is too few for text. The expense claims are fictional.
- Ask of any model: how many examples, how was it tested, and who checks the doubtful cases?

I keep: A one-page sheet: how a tree picks a question, overfitting and cross-validation, distance and k-nearest neighbours, and the honest scores from today's run.

Session 3

Measuring a model: accuracy, precision, recall, F1, ROC and AUC, thresholds, baselines, overfitting

How do I know whether a model is any good, and what should I ask the vendor who says it is?

- VENDOR QUESTION SHEET: THE NUMBERS BEHIND AN AI CLAIM
- Words: confusion matrix (TP, FP, FN, TN). Precision = TP ÷ (TP + FP). Recall (true positive rate) = TP ÷ (TP + FN). Accuracy = (TP + TN) ÷ (TP + TN + FP + FN). False positive rate = FP ÷ (FP + TN). F1 = 2 × precision × recall ÷ (precision + recall), the harmonic mean.
- ROC curve: true positive rate plotted against false positive rate for every threshold. AUC = the area under it (0.5 coin flip, 1 perfect). Cost of errors = (TP + FP) × cost of chasing a flag + FN × cost of a miss. Smoke alarm check: accuracy 96.2%, precision 32%, recall 80%, false positive rate 3.5%, F1 46%.
- 1. What test data was the figure measured on, and had the model seen that data?
- 2. How common is the thing we look for? What does the majority-class baseline score?
- 3. What are precision and recall, and at which threshold?
- 4. Who chose the threshold, and what did each kind of mistake cost?
- 5. How does it compare with a simple rule or with our current process?
- 6. How many real cases were in the test? Few cases means a shaky figure.
- 7. Can we test it on 50 of our own cases before we sign?
- 8. How will you check it after launch, as our data changes?
- 9. Who reads the false positives, and how long does it take them?
- 10. What happens to a case the model gets wrong? Who can overrule it?
- Remember: accuracy rewards doing nothing when the thing is rare. A perfect score on training data can mean overfitting.

I keep: The vendor question sheet: ten questions about the numbers behind any AI claim. Keep it next to every proposal you read.

Session 4

Words as numbers: bag of words, TF-IDF, embeddings and distance

How can a computer find the right clause or policy when I describe it in plain words?

- CHEAT SHEET: VECTORS, THE FOUR MEASURES OF CLOSENESS, AND TF-IDF
- A vector is a list of numbers in a fixed order: a = (a_1, a_2). a_1 is the first number (dimension 1), a_2 the second. Drawn as an arrow from 0: across a_1, up a_2. Direction = the mix; length = how strong.
- Symbols: Σ means add up, one term for each position i. x^2 (squared) means x × x. √ is the square root (√25 = 5). ‖a‖ is the length of a. θ (theta) is the angle between two arrows. gap_1 = the gap between the first numbers, counted as positive.
- LENGTH: $\|a\| = \sqrt{(a_1^2 + a_2^2)}$. Ana (3, 4): √(9 + 16) = 5. Any length: $\sqrt{\Sigma\, a_i^2}$.
- EUCLIDEAN DISTANCE (L2): $\sqrt{[(a_1 - b_1)^2 + (a_2 - b_2)^2]}$. Straight line between the tips. Smaller is closer. Ana to Ben 5, to Chloe 1.41.
- MANHATTAN DISTANCE (L1): $|a_1 - b_1| + |a_2 - b_2|$, where | | means the gap with its sign ignored. Walk along the grid. Smaller is closer. Ana to Ben 7, to Chloe 2.
- DOT PRODUCT: $a \cdot b = a_1b_1 + a_2b_2$ (any length: $\Sigma\, a_ib_i$). Multiply matching numbers, add. Bigger is closer; 0 means nothing shared; long arrows score higher. Ana with Ben 50, with Chloe 24, with herself 25.
- COSINE SIMILARITY: $\cos(\theta) = (a \cdot b) \div (\|a\| \times \|b\|)$. 1 same direction, 0 right angle, −1 opposite. Ignores length. Ana with Ben 50 ÷ (5 × 10) = 1.00; with Chloe 24 ÷ 25 = 0.96 (16°). Cosine distance = 1 − cos(θ).
- BAG OF WORDS: list every word once (the vocabulary), count each word in each message. "the invoice is late" = (0, 1, 1, 1, 1, 0) over send, the, invoice, is, late, rice.
- TF-IDF: tf = times the word appears in this message; df = messages containing it; N = number of messages. idf = ln(N ÷ df); ln x = how many times you multiply by e (about 2.718) to reach x. Weight = tf × idf. M06: invoice 2 × ln(40 ÷ 4) = 2 × 2.30 = 4.60.
- As in the recorded run (scikit-learn defaults): idf = ln[(1 + N) ÷ (1 + df)] + 1, weight = tf × idf, then divide the message vector by its length. M06 invoice: 2 × 3.104 ÷ 18.54 = 0.335.
- EMBEDDING: a vector produced by a trained model (all-MiniLM-L6-v2: 384 numbers, each vector length 1). Similar meanings point in similar directions. Compare with cosine.
- Remember: a score only ranks. Read the top three. Semantic search finds descriptions, exact search finds codes. Ask the vendor: which measure, and were the vectors set to length 1 first?

I keep: The cheat sheet: the four measures of closeness (Euclidean distance, Manhattan distance, dot product, cosine similarity), the length of a vector and TF-IDF, each with its formula, every symbol named and the worked numbers.

Session 5

Language models: tokens, attention and next-word probability

How does a language model turn your words into an answer, and why can the same question give different answers?

- Language model on a page
- Token, token id: the tokenizer cuts text into tokens from a fixed vocabulary (50,257 in GPT-2); the token id is the token's number in that list. The context window is how many tokens fit at once (1,024 in GPT-2).
- Logit: the raw score the model gives every possible next token. Higher means more likely; only the gaps matter, so scores can be negative.
- e and e to the x: e = 2.718..., e^x multiplies e by itself x times (e^2 = 7.39, e^0 = 1, e^-1 = 0.37). It is always positive and each extra 1 multiplies by 2.72.
- Softmax: e to each score, add them up, divide each by the total. p_i = e^(z_i) / sum_j e^(z_j), where z_i is the score of token i and the sum adds one term per token. Example: scores 2, 1, 0 give 66.5, 24.5, 9.0 percent.
- Sampling and temperature: sampling draws a token in proportion to its probability; greedy decoding always takes the top one. Temperature T divides every score first: p_i = e^(z_i/T) / sum_j e^(z_j/T). T = 2 gives 50.6, 30.7, 18.6 percent; T = 0.5 gives 86.7, 11.7, 1.6. Top-k keeps the k most probable tokens; top-p keeps the fewest top tokens adding up to p; the kept ones are divided by their total.
- Training: the Session 0 loop on text. Hide the next token, predict, measure the loss, adjust the weights by gradient descent. The text is its own label (self-supervised). Loss = -ln p(right token), ln = natural logarithm (the power of e that gives the number): -ln 0.9 = 0.11, -ln 0.18 = 1.71, -ln 0.01 = 4.61. A sentence's probability is the chain P(token 2 | token 1) x P(token 3 | tokens 1 to 2) x ...; training lowers the average loss (cross-entropy).
- Input numbers: token id picks a row of the embedding matrix (768 numbers in GPT-2), plus a row of the position table: x_t = E[id_t] + P[t].
- Self-attention: each token makes a query (what I look for), a key (what I offer) and a value (what I pass on). Score = query x key (the Session 4 dot product), softmax gives weights, output = weights x values. Attention(Q, K, V) = softmax(Q K^T / sqrt(d)) V, with K^T the keys turned on their side, d the numbers per key (64 in GPT-2, so divide by 8), later tokens blanked out. One attention head shows one pattern.
- Transformer block: self-attention plus a feed-forward network (Session 1 neurons: 768 in, a hidden layer of 3,072, 768 out). GPT-2 small stacks 12 blocks, 124 million parameters.
- Five questions for any answer: Where did this come from? Can I check it against a source? Would it change if I asked again? Does it contain figures or quotes? Who reviews it?
- Recorded runs: GPT-2 small, 124M parameters, 6, 7 and 11 November 2026.

I keep: A one-page sheet: tokens, logits, softmax, temperature, the next-token loss, self-attention and five questions to ask about any answer.

Session 6

Foundation models: pretraining, post-training, model families, reasoning and multimodal models, benchmarks

What is a foundation model, how does it become an assistant, and what does a benchmark score tell me?

- Model comparison grid
- Terms: pretraining (next-token prediction on a text corpus, a huge collection of text) gives a base model, also called a foundation model. Post-training: supervised fine-tuning (instruction tuning; data: instructions with answers people wrote) and preference tuning such as RLHF, reinforcement learning from human feedback (data: two answers and the one a person preferred; a reward model scores answers; the model is nudged towards higher scores). An LLM is a large language model.
- Average loss: add the loss $-\ln p$ of every token, divide by the number of tokens N. Example: $(1.71 + 0.69 + 1.39) \div 3 = 1.26$. Formula $L = -(1/N) \Sigma \ln p_t$, where p_t is the probability given to the true token t and Σ means add up over the tokens.
- Perplexity = e^L: as unsure as choosing among that many equally likely tokens. Blind guess among 4 words: e^1.39 = 4. Our 1.26 gives 3.5. Lower is better; 1 is perfect.
- Mixture of experts: a router scores the experts, keeps the top 2, softmax gives gate weights (e^1.2 = 3.32, e^0.6 = 1.82, so 0.65 and 0.35), output $y = \Sigma g_i E_i = 0.65 \times 4 + 0.35 \times 8 = 5.4$.
- Families: mixture of experts, reasoning model (writes working tokens first), multimodal model (photo cut into patches: 196 patches of 768 numbers in the example), diffusion model (removes noise step by step), small language model (laptop size).
- Benchmark: fixed questions, answer key, scoring rule. Accuracy = correct ÷ total (712 ÷ 1,000 = 71.2%; four-option guessing baseline 25%). Scores mislead through contamination (questions seen in training) and different settings (examples in the prompt, number of tries).
- My task (one line): _____
- Models to try (2 or 3): _____
- Ten real cases with the answer I expect: attach a list
- Three checks (for example figures, label, nothing invented): _____
- Model card questions: licence? size (parameters)? trained on? limits? what is missing?
- Benchmark claim questions: which test (MMLU, GSM8K, HumanEval, Chatbot Arena)? what setup (prompt, examples, tries)? what date? contamination checked? how does it match my task?
- Recorded runs: 6 November 2026. Qwen2.5-1.5B-Instruct and GPT-2 run locally; Claude in a fresh subagent session. Not ChatGPT or Gemini.

I keep: A one-page model comparison grid for your own work, with the questions to ask of a model card and a benchmark claim.

Session 7

Open weights, file formats, size and running cost

What does it mean to own, host or rent a model?

- RENT, HOST OR OWN: DECISION SHEET
- Task: _____ Data it touches: _____ Sensitive? (yes or no)
- Parameters = weights + biases (Session 1). GPT-2 small: 124,439,808. A bit is 0 or 1; 8 bits = 1 byte = 256 patterns; 1 GB = a billion bytes.
- Memory sum: memory ≈ N × b (N parameters, b bytes each: FP32 4, FP16 2, INT8 1, 4-bit 0.5). Example: 7 billion × 2 bytes (FP16) = 14 GB; 7 billion × 0.5 byte (4-bit) = 3.5 GB
- Quantisation in three steps: choose the scale s (step between levels), divide and round, add the zero point z. Formula q = round(x ÷ s) + z; read back $\hat{x}$ = s (q − z). Example s = 0.1, z = 8: 0.42 ⟶ q = 12 ⟶ $\hat{x}$ = 0.4, error 0.02 (at most s ÷ 2).
- Break-even volume = F ÷ (p − v): fixed cost a month ÷ (price per request if renting − running cost per request if hosting). Example 3,000,000 ÷ (1,000 − 200) = 3,750 requests a month.
- Open source (OSI definition) or open weights? Which licence? Which uses does it limit?
- File format (Safetensors, GGUF, ONNX)? Precision (FP32, FP16, INT8, 4-bit)? Quality tested on my cases?
- Route: rent / host / own / wait. Reason: _____
- One thing that would change my mind: _____

I keep: A one-page rent, host or own decision sheet, with the memory sum, the break-even sum and the licence questions to ask.

Session 8

Prompting and reasoning models

How do I get reliable work out of an assistant?

- PROMPT TEMPLATE AND LOG
- Five parts: Role. Context. Examples. Format. Constraints (and what to say when unsure).
- My prompt: _____
- Evaluation set: test cases with known answers (three to ten): _____
- Log row: Version ___ What changed ___ Runs ___ How many right ___ Accuracy = correct ÷ total ___ Next change ___
- Why a prompt works: the model continues your text one token at a time, so every word you add changes which next tokens are likely (Session 5). Temperature T divides each logit before softmax: p = e^(z/T) ÷ Σ e^(z/T), Σ meaning add up over all tokens.
- Who checks the figures before use: _____
- Template: Templates/Reusable Brief Template.docx (seven parts)

I keep: A reusable prompt template and a prompt log, so that every prompt you keep has been tested on an evaluation set.

Session 9

Giving a model your knowledge: retrieval-augmented generation, long context and fine-tuning

How do I give an AI model my company's knowledge, and when is each way worth it?

- PROMPTING, RAG OR FINE-TUNING: ask in this order.
- 1. Does a clearer prompt with the page pasted in give right answers on ten checked questions? Yes: stop.
- 2. Are the facts too many, too long or changing? Use retrieval-augmented generation (RAG). Check your chunks. Let the model say not stated.
- 3. Is the style or format still wrong after steps 1 and 2? Consider fine-tuning, with examples you own and personal data removed.
- Always: measure on the same ten questions before you add the next method.
- Formulas: cosine similarity $\cos\theta = q \cdot c / (|q||c|)$: the dot product divided by the two lengths; θ is the angle; 1 same direction, 0 unrelated, 0.6 about 53 degrees apart. Worked: (4, 3) and (3, 4) give $24 \div (5 \times 5) = 0.96$. With lengths of 1 it equals $q \cdot c$ (0.73 for clause 3.2, 0.47 for 3.1). Top-k retrieval keeps the k chunks with the largest scores. Prompt budget: prompt + retrieved text + answer ≤ context window (Q03: 94 + 104 + 300 = 498 tokens; whole handbook: 94 + 1,074 + 300 = 1,468). Matrix: a table of numbers, rows × columns. Low rank: a table rebuilt from a column B and a row A, like a times table. LoRA: $W' = W + B\,A$, with W frozen, B ($d \times r$) and A ($r \times k$), r the rank, so $r(d + k) = 8 \times (768 + 2{,}304) = 24{,}576$ trainable numbers per block instead of $d \times k = 1{,}769{,}472$; 12 blocks give 294,912.
- Terms: chunking, embedding, vector database, retriever, top-k retrieval, context window, LoRA (low-rank adaptation), adapter, rank.
- Remember: a high similarity score shows that topics are close. It does not show that the answer is present. Fine-tuning shapes style and does not reliably teach facts.
- Worksheet: six NVF requests. (a) Summarise one supplier email. (b) Answer staff questions from a handbook that changes each quarter. (c) Every customer reply ends with the house sign-off. (d) Check a price against one pasted quote. (e) Answer questions about 400 contracts. (f) Draft replies in the Sales Manager voice from 200 past emails.

I keep: A one-page decision tree: prompting first, then retrieval-augmented generation (RAG), then fine-tuning, and the test that tells you when to move on.

Session 10

When models fail, and how to check them

How do I find out when an AI answer is wrong, before it costs me?

- FIVE-STEP CHECK. The more an error would cost, the more steps you take.
- 1. Source: ask for the quote or clause, then open it.
- 2. Recount: redo every number by hand or with a formula.
- 3. Compare: check the key, or a second source.
- 4. Run twice: if the same request gives different answers, look closer.
- 5. Sign: a named person decides and records it.
- EVALUATION-SET TEMPLATE. Id | Question | Ground truth | Source | Run 1 | Run 2 | Cost of a wrong answer. Ten questions, two with no answer, kept apart from anything used to tune the tool.
- TERMS. Failure mode: a repeatable way of going wrong. Hallucination: confident output the facts do not support. Overconfidence: a sure tone whether right or wrong. Retrieval miss: the retriever fetches the wrong chunk or none. Drift: performance changes because the data or model changed. Prompt injection: text in the input that tries to steer the model. Bias: results that favour or miss some groups. Data leakage: test answers or private data reaching where they should not. Overfitting: fits the data it was tuned on and does worse on new data. Baseline: a trivial method to beat.
- FORMULAS. Accuracy = correct ÷ total; the baseline of always saying not stated scores 2 ÷ 10 = 20 percent, and three clauses score 10 ÷ 10 = 100 percent, a gain of 80 percentage points. Expected cost of an error = P(error) × cost of one error, where P(error) is the probability of an error, the share of cases that go wrong (0 to 1). Illustration: 2 of 12 totals wrong, so P(error) = 2 ÷ 12 ≈ 0.17; one error costs UGX 600,000 (invented); (2 ÷ 12) × 600,000 = UGX 100,000 a total. Check every item when a check costs less than that.
- Label right, wrong, missed, invented. Compare with the baseline of always saying not stated. Rerun after any change (drift).
- Recorded finding: most runs were right and the faults were small. Only checking found them.

I keep: A five-step check routine and an evaluation-set template (questions with ground truth), each on one page.

Session 11

Agentic AI: tools, loops, MCP, autonomy and guardrails

What changes when an assistant can act as well as answer?

- Tool Boundary Sheet (the permissions table): fill it before any agent is switched on.
- Agent: an AI model with a goal and tools. The agent loop: plan, act, observe, repeat; it stops when the goal is met, a step limit is reached or a person is needed.
- Tool use (function calling): the model writes a request; a program runs the tool and returns the result.
- Agent, built in three steps: a model that answers (prompt in, next tokens, answer out) + tools (the model writes a request, a program runs it) + a loop (the result goes back into the prompt).
- Permissions (scopes) and least privilege: list what the task needs, grant only that, like a house-sitter with the front door key only. Read and draft are enough for a mail summary.
- Risk = likelihood × impact. Illustration (invented): likelihood 0.01 (1 in 100 injected mails fools it); read only 0.01 × 0 = 0; draft 0.01 × UGX 20,000 = UGX 200; send 0.01 × UGX 20,000,000 = UGX 200,000. Same shape as expected cost in Session 10.
- MCP solves a connection count: 3 apps × 4 tools = 12 connectors without a standard; 3 + 4 = 7 pieces with one (illustration).
- Four rows: Read, Write, Send, Spend. For each: it may, a person approves, it must never.
- Human-in-the-loop approval: sends, payments, deletions and commitments always need a named person. Approval time T = n × t: 3 approvals × 2 minutes = 6 minutes in the demo mailbox.
- Guardrails: permissions, approval, an audit log line for every step, one owner, stop conditions, an off switch that works in minutes.
- Prompt injection: addressed to the AI, asks for data to leave or a deletion, asks for silence. Stop and tell the owner.
- Model Context Protocol (MCP): a standard plug (host, one client per server; servers offer tools, resources, prompts). It connects; it does not decide permissions.
- Autonomy levels: 0 answers, 1 suggests, 2 drafts and you approve, 3 acts then reports, 4 acts alone. Choose the lowest that works.

I keep: Your Tool Boundary Sheet, the permissions table for an agent: for read, write, send and spend, what it may do, what a person approves (human-in-the-loop) and what it must never do, with the owner, the audit log and the off switch.

Session 12

Workflow mapping (practical)

Which of my tasks should be a rule, an AI step or an agent?

- Workflow: a task that repeats. Workflow map: trigger, steps, handoffs (red), check (gold), one lane per person or system. Everyday example: making tea.
- AI step: one judgement by a model inside a fixed path (a spam filter). Agent: a goal, tools and a loop of plan, act, observe that picks its own next step (Session 11).
- Automation: a machine runs the steps when the trigger fires. Rule-based automation follows fixed rules, such as a bank standing order.
- Decide each step in order: clear words or numbers: rule-based automation. Meaning with a quick check: AI step. Several tools and its own next step: agent with written permissions. Money, trust or law: a person (human-in-the-loop).
- Side by side on twelve customer messages: rule 7 of 12; AI 9 exact, 11 acceptable, 1 wrong with confidence (M14). Different measures; small sample.
- Baseline: measure today first. Hours saved $H = n \times (t_0 - t_1) \div 60$: runs a month × minutes saved per run ÷ 60 (× people, if several do it). Review time is a cost.
- Payback period = one-off cost ÷ net saving a month. Net benefit = benefits − costs. ROI = net benefit ÷ costs × 100%. Kettle: payback 60,000 ÷ 20,000 = 3 months; first year benefits 12 × 30,000 = 360,000, costs 60,000 + 12 × 10,000 = 180,000, net benefit UGX 180,000, ROI 180,000 ÷ 180,000 × 100% = 100% (invented).
- NVF plan example: 4 × 80 ÷ 60 = 5.3 hours a month before review time (4 hours after 20 minutes of review a run); benefits 160,000 + 144,000 = 304,000; one-off 16 h × 30,000 = 480,000; monthly 60,000 + 40,000 + 45,000 = 145,000; net benefit 304,000 − 145,000 = UGX 159,000 a month; payback period 480,000 ÷ 159,000 ≈ 3.0 months; 12-month net 12 × 159,000 − 480,000 = UGX 1,428,000; first-year ROI 1,428,000 ÷ 2,220,000 ≈ 64% (invented inputs).
- Assistant sketch: trigger, inputs, outputs, fewest permissions, who approves, the off switch.
- Measure four weeks: minutes each run with review, errors and their cost, cases sent to a person, your stop rule.
- Templates: Task Card, Use-Case Scorecard, Tool Boundary Sheet (permissions table), Business Case Workbook (Inputs sheet).

I keep: Your mapped workflow, the choice for each step, your assistant sketch, and the baseline and hours-saved estimate in the Business Case Workbook.

Session 13

Governance and buying (practical)

What rules must my company set for AI, and how do I question what I am sold?

- AI governance: the policies, roles and checks that direct and control AI use. Everyday twin: the rules for lending your car (policy, risk list, classification, ask first, logbook, check the garage).
- Acceptable use policy (the AI Use Policy template): eleven sections. I drafted 3 approved tools, 4 data classification rules, 5 human oversight, 7 accounts, 8 incidents, 10 review date.
- Data classification, four questions: living person (personal data)? harmful if leaked? staff only? already published? The strictest answer wins.
- Risk register: one row per risk with score, level, owner, control and residual risk. Risk score R = L × I, likelihood (1 to 5) times impact (1 to 5), a risk matrix. Flat tyre: 4 × 2 = 8, Medium; with a spare tyre 4 × 1 = 4, Low. High 15 to 25, Medium 8 to 14, Low 1 to 7. A control lowers likelihood or impact.
- Every limit of a model gives a rule: human oversight, approval on every send plus an audit log, data classification, licence read, least privilege, re-run the evaluation set after a change.
- Total cost of ownership (TCO) = set-up + months × monthly fee, plus usage. Phone: 0 + 24 × 100,000 = UGX 2,400,000. OrbitDesk: 0 + 36 × USD 2,400 = USD 86,400 before usage.
- Vendor red flags: no number, no approval, trains on my data, no references, no cap, no exit, a deadline, bank login.
- Vendor due diligence, five steps: ask in writing, score 0 to 2, ask for proof, pilot with a stop rule, sign with an exit.
- Law, my understanding: the Data Protection and Privacy Act 2019 covers personal data (sections 19, 20, 21, 23, 29). Ask my lawyer.
- Frameworks: NIST AI RMF (voluntary), ISO/IEC 42 001 (an AI management system standard a vendor can be certified against), OECD AI Principles, African Union Continental AI Strategy, EU AI Act (only if I serve the EU).

I keep: Your Risk Register, your AI Use Policy draft, your Vendor Question List with five questions, and your page of questions for the lawyer.

Session 14

Data readiness assessment, roadmap and 90-day plan (practical)

How ready is my organisation's data for AI, what will I fix first, and who will own it?

- Everyday twin: a pantry audit. Data inventory (the list), metadata (the labels), lineage (where it came from), data governance (who may open the cupboard), quality dimensions (the checks), roadmap (the shopping plan).
- Six data quality dimensions: completeness, accuracy, consistency, timeliness, uniqueness, validity. Completeness = filled values ÷ all values × 100% (contact list: 4 ÷ 5 = 80%; NVF PO numbers: 58 ÷ 60 = 96.7%). Uniqueness = distinct records ÷ all records (contact list: 4 ÷ 5 = 80%, duplicate rate 20%; NVF: 59 ÷ 60 = 98.3%).
- Five kinds of data: tables (structured), text, images, audio, sensor data. First steps: define columns, find the current version, read images with OCR, transcribe audio, check gaps.
- Readiness scorecard: Inventory, Readable, Complete and unique, Metadata, Rights, Current. Green 2, Amber 1, Red 0. Readiness score = Σ points (Σ: add up, one term per area), out of 12 (NVF: 2+1+1+1+0+1 = 6). A Red on rights stops the work.
- Six steps: inventory, digitise, clean, describe, protect, index and test. Start with one folder and one task.
- First test: an evaluation set of 20 questions with known answers, two or three traps, a line for success written first. Chunking splits text into passages; an embedding is a vector for meaning.
- Every roadmap action: owner, start, due date, hours, depends on, a success measure (a KPI) someone else can check. Total effort = Σ hours (NVF: 4+6+8+5+10+2 = 35). KPI change = (new − old) ÷ old × 100%: score 6 to 9 is 3 ÷ 6 = +50%; faults 8 to 0 is −8 ÷ 8 = −100%.
- Check-in milestones: day 30 (controls in place? gap 1 closed?), day 60 (stop rule met? actions on time?), day 90 (re-score, then continue, expand, pause or stop).
- Within 24 hours: book the three dates in my diary and tell each owner their action.
- The follow-up kit: 30-60-90 Check-in Form, Results Tracker, Sponsor Briefing.

I keep: Your inventory and scorecard, your roadmap with its risks and decisions, the Follow-up Kit (Check-in Form, Results Tracker, Sponsor Briefing) and your answer to the Session 0 question.